

Table of Contents

1.0	Abstract	1
2.0	Introduction	2
3.0	Problem Description	2
3.1	Assumptions.....	3
4.0	How To Think About The Problem	3
5.0	Program Code	4
5.1	A* Algorithm Pseudocode	4
5.21	A* Pseudocode Simplified Version.....	5
5.2	BFS Pseudocode	6
6.0	C++ Code Implementation	7
6.1	A* Code Implementation.....	7
6.2	BFS Algorithm	10
7.0	Complexity Analysis	13
7.1	Time Complexity	13
7.11	A* Algorithm	13
7.12	BFS Algorithm.....	13
7.2	Space Complexity.....	14
7.21	A* Algorithm	14
7.22	BFS Algorithm.....	14
8.0	Algorithms Comparisons	14

8.1 A* Algorithm	14
8.2 BFS Algorithm	15
8.3 Comparison Table.....	16
9.0 Conclusion.....	16
10.0 References.....	17

1.0 Abstract

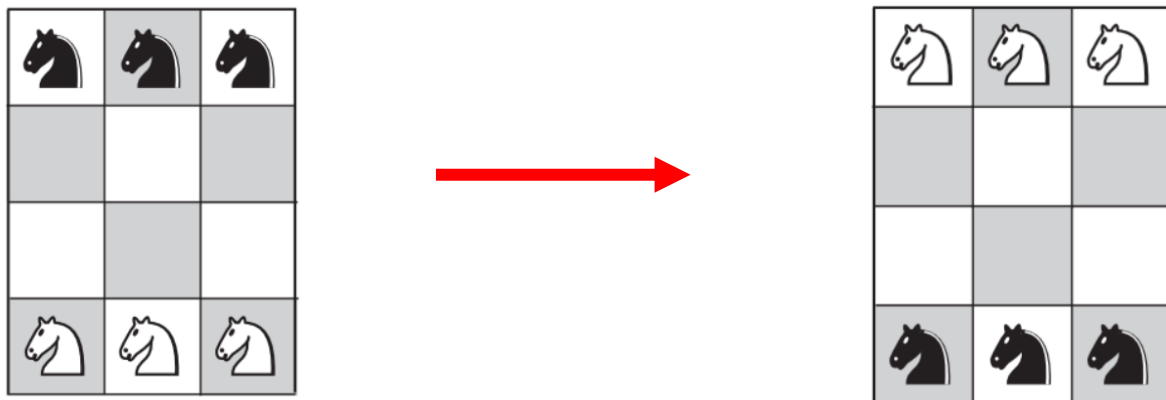
This report presents a solution to the Knight Exchange Problem using both the A* search algorithm and Breadth-First Search (BFS). The goal is to swap white and black knights on a limited 4×3 chessboard through valid knight moves requiring the least number of moves. We implement and compare two search strategies: (1) an informed heuristic search using A* with path cost and a heuristic approximation of remaining misplacement to direct the search, and (2) an uninformed BFS approach that systematically explores all possible moves. This report describes the problem formulation, algorithm design, pseudocode, implementation details, complexity analysis, and provides a comprehensive comparison between the two approaches to determine the most efficient solution strategy for problems of varying complexity.

Design and Analysis of Algorithms

Task 4

2.0 Introduction

Search algorithms are used by Artificial intelligence to overcome problems such as pathfinding and optimization. It involves solving a challenging combinatorial problem, because knights have to be moved in precise location as it was to make a position exchange. However, because of the unique constraints of the knight move, the problem possesses a non-obvious search space, and compensates with an invocation of sophisticated algorithms to seek an optimal solution. For this challenge, I selected two different algorithms to deal with it, namely: A informed search algorithm **A*** is a search strategy that is driven by its heuristic evaluation to support exploration. Breadth-First Search (**BFS**) is a wide searching manner that investigates the entire search space completely and becomes optimal without the absence of heuristic information. Through this method we are able to see the relative strengths of informed and uninformed search algorithms when used on a variety of difficult scenarios.



3.0 Problem Description

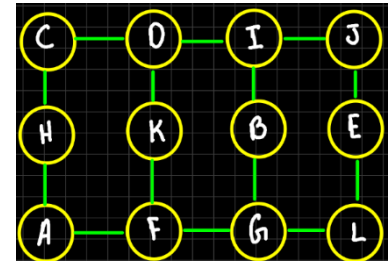
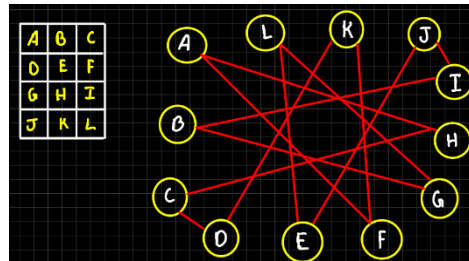
There are six knights on a 3×4 chessboard: the three white knights are at the bottom row, and the three black knights are at the top row. Design an iterative improvement algorithm to exchange the knights to get the position shown on the right of the figure in the minimum number of knights moves, not allowing more than one knight on a square at any time.

Design and Analysis of Algorithms

Task 4

3.1 Assumptions

- The initial positions are: Black pieces on top row (0th row) and the white knights in the bottom row (3rd row).
- Empty cells are represented by '.' (dots).
- Knights move according to chess rules (L-shaped: 8 possibilities).
- Only valid knight moves are allowed; no diagonal, horizontal or vertical direct movements.
- Every move is basically swapping a knight with an empty square such that movement is legal.



4.0 How To Think About The Problem

Step 1: Start with a feasible solution. Which means just reach a valid solution , not the best , that just follows the rules.

Step 2: Know each square of the grid is connected to which square. Look at the picture below to understand For example, square “A” can go to square “F” or square “H”. We can simplify this network into the connection shown below.

The picture clearly shows the connection between the squares. From the picture, we can understand that squares “H” is reachable from square “C” or “A” and so on. Now the first step in my code, I need to make a 3x4 grid and specify the connection of each square.

Second thing we have to do in the code, we must clarify the goal of each square. This can easily visualized in the picture shown in the previous page.

- Goal of White: A B C
- Goal of Black: J K L

Design and Analysis of Algorithms

Task 4

5.0 Program Code

5.1 A* Algorithm Pseudocode

function solve_with_A_star(start_board):

Initialize empty priority queue PQ

Initialize empty visited set

Initialize best_solution to null

Initialize min_steps to infinity

Calculate $h(\text{start_board})$ = number of knights not in goal row

Insert $(f(\text{start_board}), \text{start_board})$ into PQ where $f(n) = g(n) + h(n)$

While PQ is not empty:

current_board = Extract-Min from PQ

If current_board is already visited:

Continue

Add current_board to visited

If current_board is goal state:

If current_board.moves.length < min_steps:

min_steps = current_board.moves.length

best_solution = current_board

Continue

For each knight position (i,j) on current_board:

For each possible knight move direction (dx,dy) :

Design and Analysis of Algorithms

Task 4

`new_position = (i+dx, j+dy)`

If `new_position` is valid and empty:

`next_board = copy of current_board`

Swap knight at `(i,j)` with empty space at `new_position`

`next_board.g = current_board.g + 1`

`next_board.moves.append(Move(i,j,i+dx,j+dy))`

Calculate `h(next_board)`

`f(next_board) = next_board.g + h(next_board)`

Insert `(f(next_board), next_board)` into PQ

Return `best_solution`

5.21 A* Pseudocode Simplified Version

- Initialize the board with the start configuration.
- Define a **heuristic**: the number of knights not in their target row.
- Use a **priority queue** to explore the board with the lowest cost first.
- For each board, check all knight positions and attempt valid knight moves.
- If a goal state is reached with fewer steps than the previously found solutions, update the best result.
- At the end, display the sequence of moves to reach the goal.

Design and Analysis of Algorithms

Task 4

5.2 BFS Pseudocode

function solve_with_BFS(start_board):

Initialize empty queue Q

Initialize empty visited set

Insert start_board into Q

Add start_board to visited

While Q is not empty:

current_board = Dequeue from Q

If current_board is goal state:

Return current_board as solution

For each knight position (i,j) on current_board:

For each possible knight move direction (dx,dy):

new_position = (i+dx, j+dy)

If new_position is valid and empty:

next_board = copy of current_board

Swap knight at (i,j) with empty space at new_position

next_board.moves.append(Move(i,j,i+dx,j+dy))

If next_board not in visited:

Add next_board to visited

Insert next_board into Q

Return "No solution found"

Design and Analysis of Algorithms

Task 4

6.0 C++ Code Implementation

6.1 A* Code Implementation

```
#include <iostream>
#include <vector>
#include <queue>
#include <set>
#include <algorithm>

using namespace std;

const int ROWS = 4;
const int COLS = 3;

int dx[] = {1, 2, 2, 1, -1, -2, -2, -1};
int dy[] = {2, 1, -1, -2, -2, -1, 1, 2};

struct Move {
    int fromX, fromY;
    int toX, toY;
};

struct Board {
    vector<vector<char>> grid;
    vector<Move> moves;
    int g;

    Board() {
        grid = vector<vector<char>>(ROWS, vector<char>(COLS, '.'));
        g = 0;
    }

    bool operator<(const Board& other) const {
        return grid < other.grid;
    }
};

int heuristic(Board& b) {
    int wrong = 0;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (b.grid[i][j] == 'W' && i != 0) wrong++;
            if (b.grid[i][j] == 'B' && i != 3) wrong++;
        }
    }
}
```

Design and Analysis of Algorithms

Task 4

```
    }
    }
    return wrong;
}

bool inside(int x, int y) {
    return x >= 0 && x < ROWS && y >= 0 && y < COLS;
}

bool isGoal(Board& b) {
    return b.grid[0][0] == 'W' && b.grid[0][1] == 'W' && b.grid[0][2] == 'W' &&
           b.grid[3][0] == 'B' && b.grid[3][1] == 'B' && b.grid[3][2] == 'B';
}

void printBoard(Board& b) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            cout << b.grid[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

void solve(Board start) {
    auto cmp = [](pair<int, Board> a, pair<int, Board> b) {
        return a.first > b.first;
    };

    priority_queue<pair<int, Board>, vector<pair<int, Board>>, decltype(cmp)>
pq(cmp);
    set<vector<vector<char>>> visited;

    pq.push({heuristic(start), start});

    Board best;
    int minSteps = 1e9;

    while (!pq.empty()) {
        Board cur = pq.top().second;
        pq.pop();

        if (visited.count(cur.grid)) continue;
        visited.insert(cur.grid);
```

```

    if (isGoal(cur)) {
        if (cur.g < minSteps) {
            minSteps = cur.g;
            best = cur;
        }
        continue;
    }

    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (cur.grid[i][j] == '.') continue;

            for (int d = 0; d < 8; d++) {
                int ni = i + dx[d];
                int nj = j + dy[d];

                if (inside(ni, nj) && cur.grid[ni][nj] == '.') {
                    Board next = cur;
                    swap(next.grid[i][j], next.grid[ni][nj]);
                    next.g = cur.g + 1;
                    next.moves.push_back({i, j, ni, nj});
                    int cost = next.g + heuristic(next);
                    pq.push({cost, next});
                }
            }
        }
    }
}

cout << "Best path with " << best.moves.size() << " moves:\n";
Board temp = start;
printBoard(temp);

for (auto m : best.moves) {
    cout << "Move from (" << m.fromX << ", " << m.fromY << ") to (" << m.toX
<< ", " << m.toY << ")\n";
    swap(temp.grid[m.fromX][m.fromY], temp.grid[m.toX][m.toY]);
    printBoard(temp);
    cout << "-----\n";
}

cout << "Total moves: " << best.moves.size() << endl;
}

```

```
int main() {  
    Board start;  
    start.grid[0][0] = 'B';  
    start.grid[0][1] = 'B';  
    start.grid[0][2] = 'B';  
    start.grid[3][0] = 'W';  
    start.grid[3][1] = 'W';  
    start.grid[3][2] = 'W';  
  
    solve(start);  
}
```

6.2 BFS Algorithm

```
#include <iostream>  
#include <vector>  
#include <queue>  
#include <set>  
#include <algorithm>  
  
using namespace std;  
  
const int ROWS = 4;  
const int COLS = 3;  
  
int dx[] = {1, 2, 2, 1, -1, -2, -2, -1};  
int dy[] = {2, 1, -1, -2, -2, -1, 1, 2};  
  
struct Move {  
    int fromX, fromY;  
    int toX, toY;  
};  
  
struct Board {  
    vector<vector<char>> grid;  
    vector<Move> moves;  
  
    Board() {  
        grid = vector<vector<char>>(ROWS, vector<char>(COLS, '.'));  
    }  
  
    bool operator<(const Board& other) const {  
        return grid < other.grid;  
    }  
};
```

Design and Analysis of Algorithms

Task 4

```

    }
};

bool inside(int x, int y) {
    return x >= 0 && x < ROWS && y >= 0 && y < COLS;
}

bool isGoal(Board& b) {
    return b.grid[0][0] == 'W' && b.grid[0][1] == 'W' && b.grid[0][2] == 'W' &&
           b.grid[3][0] == 'B' && b.grid[3][1] == 'B' && b.grid[3][2] == 'B';
}

void printBoard(Board& b) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            cout << b.grid[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

void solve(Board start) {
    queue<Board> q;
    set<vector<vector<char>>>> visited;

    q.push(start);
    visited.insert(start.grid);

    while (!q.empty()) {
        Board cur = q.front();
        q.pop();

        if (isGoal(cur)) {
            cout << "Solution found with " << cur.moves.size() << " moves:\n";
            Board temp = start;
            printBoard(temp);

            for (auto m : cur.moves) {
                cout << "Move from (" << m.fromX << "," << m.fromY << ") to (" <<
m.toX << "," << m.toY << ")\n";
                swap(temp.grid[m.fromX][m.fromY], temp.grid[m.toX][m.toY]);
                printBoard(temp);
                cout << "-----\n";
            }
        }
    }
}

```

Design and Analysis of Algorithms

Task 4

```

    }

    cout << "Total moves: " << cur.moves.size() << endl;
    return;
}

for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        if (cur.grid[i][j] == '.') continue;

        for (int d = 0; d < 8; d++) {
            int ni = i + dx[d];
            int nj = j + dy[d];

            if (inside(ni, nj) && cur.grid[ni][nj] == '.') {
                Board next = cur;
                swap(next.grid[i][j], next.grid[ni][nj]);

                if (!visited.count(next.grid)) {
                    next.moves.push_back({i, j, ni, nj});
                    visited.insert(next.grid);
                    q.push(next);
                }
            }
        }
    }
}

cout << "No solution found!" << endl;
}

int main() {
    Board start;
    start.grid[0][0] = 'B';
    start.grid[0][1] = 'B';
    start.grid[0][2] = 'B';
    start.grid[3][0] = 'W';
    start.grid[3][1] = 'W';
    start.grid[3][2] = 'W';

    solve(start);

    return 0; }

```

Design and Analysis of Algorithms

Task 4

7.0 Complexity Analysis

7.1 Time Complexity

7.11 A* Algorithm

Worst-case: $O(b^d)$, where:

- b is the branching factor i.e. no more than 48 when considering 6 pieces and 8 knight moves per piece
- d is the depth of the shortest path solution

Average-case: $O(b^d \times \log(b^d))$ due to priority queue operations

The heuristic guidance provides to reduce the branching factor which effectively. Observing every node has $O(\log V)$ operations in the priority queue. With (12 choose 6), we approximate the number of unique board states to be about 924 – at most, this sets the limits of the search space.

7.12 BFS Algorithm

Worst-case: $O(b^d)$, where:

- b is the branching factor (≤ 48)
- d is the depth of the shortest path solution

Average-case: $O(b^d)$, with all nodes at depth d being explored

Each node examination requires constant time for queue operations BFS has been designed to find the best path with least moves.

Design and Analysis of Algorithms

Task 4

7.2 Space Complexity

7.21 A* Algorithm

Priority queue size: $O(b^d)$

Visited set size: $O(V)$, where V is the number of unique board states

Total: $O(b^d + V)$

7.22 BFS Algorithm

Queue size: $O(b^d)$

Visited set size: $O(V)$

Total: $O(b^d + V)$

8.0 Algorithms Comparisons

8.1 A* Algorithm

A* is a mixed informed search strategy using features of uniform-cost search and Greedy best-first search. It uses a cost function $f(n) = g(n) + h(n)$ if it is defined as below:

$g(n)$ is the total cost that has built up from start to node n .

$h(n)$ is an estimate of how far node n is, from the target goal state (by the heuristic)

As for our implementation, the selected heuristic states that we need to determine the number of knights that are not in their intended rows. Since this heuristic is admissible (never overestimate the path to destination or perhaps equal to true cost to the target) and consistent (abides by triangle inequality), the A* algorithm is guaranteed to find the optimal route.

➤ Key Characteristics:

- **Informed Search:** Uses problem-specific knowledge to guide the search

Design and Analysis of Algorithms

Task 4

- **Optimality:** Guarantees that when an admissible heuristic is used, the optimal results are obtained.
- **Efficiency:** Significantly reduces the amount of states that have to be observed compared to unskilled methods.
- **Completeness:** Will always find a solution if one exists
- **Memory Usage:** Uses both a priority queue and a visited set, and they may consume a lot of memory.

For complex problem domains, A* can do better than uninformed search because it narrows its resources to relevant states instead of examining the infinitely many non-goal states that uninformed methods could inspect.

8.2 BFS Algorithm

Breadth-First Search exhaustively explores each node on the current depth and only then moves to the nodes on the depth below. The search algorithm uses a simple FIFO queue to handle its frontier.

➤ Key Characteristics:

- **Uninformed Search:** Does not use problem-specific knowledge
- **Optimality:** Will always find out the path of least moves if it presumes equal action costs.
- **Completeness:** Will always find a solution if one exists
- **Memory Usage:** Maintains a queue that can grow exponentially with depth

8.3 Comparison Table

Feature	A* Search	Breadth-First Search (BFS)
Search Type	Informed	Uninformed
Uses Heuristic	Yes	No
Optimality	Yes (with admissible heuristic)	Yes (in uniform step-cost environments)
Completeness	Yes	Yes
Time Efficiency	Higher (guided by heuristic)	Lower (explores blindly)
Space Complexity	High (due to priority queue and visited states)	Very High (due to full tree expansion)
Suitability for Current Task	Excellent	Acceptable, but inefficient

9.0 Conclusion

I successfully performed and analyzed the efficiency of using both A* and BFS algorithms to solve the Knight Exchange Problem in this project. From my in-depth analysis of both algorithms and their actual implementation, I come to the following conclusions:

Design and Analysis of Algorithms

Task 4

The two algorithms effectively provide optimal exchange strategies for the Knight Exchange Problem. With larger scale issues utilizing A* search, the woes of the exhaustive search are alleviated by exploration based on heuristics, and A* is therefore encouraged to be used in various sophisticated chess puzzles as well as handling large number of states. For more or less minor versions of the issue, BFS is not only easy to implement but it also delivers best results; however, efficiency plummets significantly with increased scope of the problem, which results in a dramatic increase of time and memory usage. Running either A* or BFS has several unique challenges: When applied to large problem domains, however, A* tends to perform better as a result. In fact, A* can sometimes be more complicated to apply though. When modified for the 4x3 Knight Exchange Problem, A* shows much greater scalability than BFS, even though both algorithms can find solutions. By using the knight counts in the wrong place heuristic in our A* approach, we establish admissibility of the heuristic, slashes the number of examined nodes, and preserves optimality in the results. Based on these outcomes, we reinforce the importance of considering problems carefully before selecting algorithms and present evidence of how informed search strategies beat algorithms for discrete combinatorial problems. The conclusions of this research prove useful for solving analogous issues in various spheres of AI related to pathfinding and state-space search.

10.0 References

- <https://www.unipuzzle.com/chess-puzzles/chess.php?name=six-knights-guarinis-problem> (to simulate the problem and try)
- https://www.youtube.com/watch?v=EuDnVV8j_Zs (helpful tips on how to think about the problem)
- <https://www.geeksforgeeks.org/a-search-algorithm/>
- [Guarini's Puzzle, A Less Elegant Solution | Mostafa Touny](#)
- <https://puzzling.stackexchange.com/questions/25287/switch-the-knights/25301#25301>